

Expressions mathématiques lisibles dans un PDF

Guide reproductible — chaîne de rendu du projet Hakili Lab (correction assistée par IA)
16 juillet 2026 — **ce document a été généré avec la chaîne décrite ci-dessous** : les exemples de la section 6 sont calculés par le vrai code au moment de la génération.

1. Le problème

Les textes produits par les LLM (corrections, diagnostics, exercices) écrivent les mathématiques en **notation informatique** : `sqrt(25)`, `x**2`, `<=`, `(3/4)`, voire du LaTeX brut (`\frac{7}{12}`) — même quand le prompt l'interdit. Rendu tel quel dans un PDF, c'est illisible pour un élève ou un enseignant. Et la solution « évidente » (LaTeX/MathJax) est lourde : elle impose une distribution TeX ou un navigateur headless.

Notre approche : rester en pur Python, convertir la notation informatique en **symboles Unicode + balises HTML** `<sup>/<sub>` par regex, et générer le PDF depuis du HTML. Niveau collège (6e–3e), cela couvre tout le besoin : racines, puissances, fractions, indices, comparaisons, ensembles, géométrie.

2. La chaîne de rendu et les outils

Outil	Version	Rôle
Jinja2	≥ 3.x	Template HTML du rapport (autoescape actif ; les fragments maths convertis sont marqués Markup)
xhtml2pdf	≥ 0.2.14	Conversion HTML+CSS → PDF (<code>pisa.CreatePDF</code>), pur Python
ReportLab	(dépendance de xhtml2pdf)	Moteur PDF sous-jacent + enregistrement des polices TTF Unicode
MarkupSafe	≥ 2.x	Marquage des fragments HTML sûrs (Markup)
re (stdlib)	—	Toute la conversion mathématique : un module de helpers purs, sans dépendance

Flux : **texte LLM** → **normalisation regex** → **template Jinja2** → **HTML+CSS** → **xhtml2pdf** → **PDF**. Aucun LaTeX, aucun navigateur, aucune dépendance système : `pip install xhtml2pdf jinja2` suffit.

Décision de fond : on ne cherche PAS à rendre du LaTeX. On se limite au répertoire de symboles **WGL4** (socle garanti par les polices Windows/Linux courantes : $\leq \geq \neq \approx \pm \times \div - \sqrt{\infty}^\circ \pi \rightarrow \equiv$, grec de base) plus `<sup>/<sub>` pour exposants, indices et fractions. Tout symbole hors de ce socle est remplacé par du français lisible (section 5). Résultat : zéro carré « tofu » dans le PDF, quel que soit le poste.

3. Étape 1 — Contraindre le LLM en amont (prompt)

Première ligne de défense : imposer une notation ASCII **simple et régulière** dans le prompt système de chaque tâche (correction, diagnostic, remédiation). Une notation régulière se convertit ensuite par regex de façon fiable :

NOTATION MATHÉMATIQUE (obligatoire) :

- Écris les maths en ASCII simple, JAMAIS en LaTeX ni en Markdown.
- Racine : $\sqrt{x}$ Puissance : x^2 Fraction : (a/b)
- Indice : u_1 Pi : π Comparaison : $\leq$ $\geq$ $\neq$

Les LLM fuient malgré tout (LaTeX résiduel, ** Python, gras Markdown) : c'est pour cela que la conversion aval rattrape AUSSI ces formes — **défense en profondeur**. Ne comptez jamais sur le prompt seul.

4. Étape 2 — Conversion regex en deux passes

Un seul module de ~140 lignes, sans dépendance (regex stdlib uniquement), partagé entre le rendu PDF et l'interface web. Deux fonctions publiques : `ascii_math_upgrade()` et `math_to_html()`.

Passé A — notations informatiques → symboles Unicode (AVANT échappement HTML)

Cette passe traite $\leq$, $\geq$, ... qui contiennent des caractères HTML : elle doit donc s'exécuter **avant** l'échappement de $\&$ $<$ $>$.

```
def ascii_math_upgrade(s: str) -> str:
    """Notations informatiques -> symboles Unicode (AVANT échappement HTML,
    car  $\leq$  et  $\geq$  contiennent des caractères HTML)."""
    # 1. LaTeX résiduel -> formes ASCII gérées en aval (défense en profondeur)
    s = re.sub(r'\\[dt]?frac{([^{]+)}{([^{]+)}\\}', r'(\1/\2)', s)
    s = re.sub(r'\\sqrt{([^{]+)}\\}', r'sqrt(\1)', s)
    for cmd, sym in [(r'\times', 'x'), (r'\cdot', 'x'), (r'\pi', '\pi'),
                     (r'\leq', '\leq'), (r'\geq', '\geq'), (r'\neq', '\neq')]:
        s = s.replace(cmd, sym)
    # 2. Comparaisons/flèches - L'ORDRE COMPTE :  $\leq$  avant  $\leq$  et  $\geq$ 
    s = re.sub(r's* $\leq$ s*', ' équivalent à ', s)
    s = re.sub(r's* $\leq$ s*', '  $\leq$  ', s)
    s = re.sub(r's* $\geq$ s*', '  $\geq$  ', s)
    s = re.sub(r's*(?:=>|->)\s*', '  $\rightarrow$  ', s)
    s = re.sub(r's*(?:=|!=)\s*', '  $\neq$  ', s)
    # 3. Puissance Python 2**3 -> 2^3 (rendue en  $\lt sup \gt$  à la passe B)
    s = s.replace('**', '^')
    # 4. ~ d'approximation entre deux valeurs ->  $\approx$ 
    s = re.sub(r'(?=[\s\d])~(?=[\s\d])', '  $\approx$  ', s)
    # 5. Multiplication "" entre deux termes ->  $\times$ 
    s = re.sub(r'(?=[A-Za-z0-9])\*(?=[A-Za-z0-9])', '  $\times$  ', s)
    s = re.sub(r'(?=[A-Za-z0-9])\s+\*(?=[A-Za-z0-9])', '  $\times$  ', s)
    return s
```

Passé B — structures (racines, exposants, fractions, indices) → HTML

Après échappement HTML, on insère les balises $\lt sup \gt$ / $\lt sub \gt$. L'échappement systématique AVANT insertion des balises est ce qui permet de marquer le résultat Markup sans risque : aucun HTML brut du LLM ne peut passer.

```
_SUP = re.compile(r'^(?=[^{}]*\\|\\([{}]*\\)|[+-]?[A-Za-z0-9]+|[+-])')
_SQRT = re.compile(r'sqrt(((?:[{}]*\\|\\([{}]*\\))*\\)', re.IGNORECASE)
_FRAC = re.compile(r'((\d+)/(\d+))')
# Fraction nue "7/12" : jamais adjacente à un chiffre, un slash, un point
# ou une virgule -> protège les dates (06/07/2026) et les décimaux.
_FRAC_BARE = re.compile(r'(?![\d/.,_])(\d{1,3})/(\d{1,4})(?![\d/])')
# Indice : LETTRE ISOLÉE + underscore ("u_1" oui, "copy_id" non)
_SUB = re.compile(r'\b([A-Za-z])_([A-Za-z0-9]{1,2})\b')

def math_to_html(s: str) -> str:
    s = ascii_math_upgrade(s) # passe A
    s = s.replace('&', '&amp;').replace('<', '&lt;').replace('>', '&gt;')
    def _sup(m):
```

```

    inner = m.group(1).strip('{}()')
    return f'<sup>{inner}</sup>'
# sqrt(expr) -> √expr, ou √(expr) si expression composée
def _sqrt(m):
    inner = _SUP.sub(_sup, m.group(1).strip())
    return f'√{inner}' if inner.isalnum() else f'√({inner})'
s = _SQRT.sub(_sqrt, s)
# Exposants AVANT fractions : dans "2^3/2^5", le 3/2 ne doit
# pas devenir une fraction.
s = _SUP.sub(_sup, s)
frac = lambda m: (f'<sup>{m.group(1)}</sup>'
                  f'&fracl; <sub>{m.group(2)}</sub>')
s = _FRAC.sub(frac, s)
s = _FRAC_BARE.sub(frac, s)
s = _SUB.sub(r'\1<sub>\2</sub>', s)          # u_1 -> u indice 1
s = re.sub(r'\bpi\b', 'π', s)
return s  # à marquer Markup(...) côté template Jinja2

```

L'ordre des règles est la partie délicate — trois pièges vécus :

- $\Leftrightarrow$ doit être traité avant $\leq$ et $\geq$, sinon « $a \Leftrightarrow b$ » devient « $a \leq b$ ».
- Les exposants avant les fractions : dans $2^3/2^5$, le « $3/2$ » ne doit pas devenir une fraction.
- Les fractions nues ($7/12$) exigent des gardes : jamais adjacentes à un chiffre, un slash, un point ou une virgule — sinon les dates (06/07/2026), les couples d'années et les décimaux sont détruits. De même l'indice x_A exige une lettre isolée, sinon `copy_id` devient « `copyid` ».

5. Étape 3 — Garde-fou police : le filtre WGL4

Le LLM peut aussi émettre directement des symboles Unicode exotiques que les polices PDF ne couvrent pas : l'appartenance $\in$, la double flèche $\Leftrightarrow$, les ensembles de nombres « double barre » (U+211D pour R, U+2115 pour N), les flèches combinantes de vecteurs... Ils s'affichent alors en carrés « tofu ». Avant tout rendu, un remplacement mappe chaque symbole **hors WGL4** vers du français lisible par un élève — jamais vers du jargon ASCII :

Point important vérifié empiriquement : ce filtre reste obligatoire **même avec une police complète**. Segoe UI Symbol contient bien le glyphe U+211D (R double barre) — ReportLab seul le dessine correctement — mais la chaîne `xhtml2pdf` le remplace quand même par un carré. Ne vous fiez donc pas aux tables de couverture de la police : testez le rendu final, et filtrez tout ce qui est hors WGL4.

```

# Symboles HORS répertoire WGL4 -> remplacement lisible par un élève.
# WGL4 = socle commun des polices courantes : ≤ ≥ ≠ ≈ ± × ÷ − √ ∞ ° π → ≡
# et le grec de base y figurent -> CONSERVÉS tels quels.
# Clés notées en échappements \uXXXX : ces symboles ne passent justement
# pas dans toutes les polices (y compris celle de ce bloc de code !) et
# les échappements survivent à tout copier-coller.
_MATH_SAFE = {
    "\u2115": "N", "\u2124": "Z",      # ensembles N et Z (double barre)
    "\u211a": "Q", "\u211d": "R",      # ensembles Q et R (double barre)
    "\u20d7": "",                      # flèche combinante de vecteur AB
    "\u21d2": " → ",                   # double flèche d'implication
    "\u21d4": " équivaut à ",          # double flèche d'équivalence
    "\u2208": " appartient à ", "\u2209": " n'appartient pas à ",
    "\u2282": " inclus dans ", "\u222a": " U ", "\u2229": " inter ",
    "\u22c5": " × ",                  # point de multiplication
    "\u22a0": "angle ", "\u22a5": " perpendiculaire à ",
    "\u2225": " // ",                 # parallèles
}

```

```
# Exposants Unicode (2 puissance -2, etc.) traités par SÉQUENCE :
# "2\u207b\u2072" -> "2^-2" -> UN SEUL <sup>-2</sup> au rendu.
_SUP_CHARS = "".join(map(chr, [0x2070, 0xB9, 0xB2, 0xB3,
                               *range(0x2074, 0x207A), # 4 à 9
                               0x207F, 0x207A, 0x207B])) # n + -
_SUPER_MAP = str.maketrans(_SUP_CHARS, "0123456789n+-")
_SUPER_RUN = re.compile(f"[{_SUP_CHARS}]+")

def _safe_text(s: str) -> str:
    s = _SUPER_RUN.sub(lambda m: "^" + m.group(0).translate(_SUPER_MAP), s)
    for char, repl in _MATH_SAFE.items():
        s = s.replace(char, repl)
    return re.sub(r" {2,}", " ", s)
```

6. Démonstration — calculée par le vrai code à la génération de ce PDF

Chaque ligne ci-dessous est passée dans le pipeline réel (`_safe_text` puis `math_to_html`) au moment où ce document a été généré :

Entrée brute (telle qu'écrite par le LLM)	Rendu dans le PDF
<code>sqrt(49) = 7</code>	$\sqrt{49} = 7$
<code>sqrt(a^2 + b^2)</code>	$\sqrt{a^2 + b^2}$
<code>x^2 - 5x + 6 = 0</code>	$x^2 - 5x + 6 = 0$
<code>2*10 = 1024</code>	$2^{10} = 1024$
<code>(3/4) + (1/6) = 11/12</code>	$\frac{3}{4} + \frac{1}{6} = \frac{11}{12}$
<code>7/12 des eleves ont reussi</code>	$\frac{7}{12}$ des eleves ont reussi
<code>Test du 06/07/2026 : note 12,5/20</code>	Test du 06/07/2026 : note 12,5/20
<code>a <= b => a + c <= b + c</code>	$a \leq b \rightarrow a + c \leq b + c$
<code>x != 0 et u_n = u_1 + (n-1) * r</code>	$x \neq 0$ et $u_n = u_1 + (n-1) \times r$
<code>\frac{22}{7} ~ 3.14</code>	$\frac{22}{7} \approx 3.14$
<code>\sqrt{2} \times \pi</code>	$\sqrt{2} \times \pi$
<code>Aire = pi * R2 = 3,14 * 25 cm2</code>	$\text{Aire} = \pi \times R^2 = 3,14 \times 25 \text{ cm}^2$
<code>2■² = 1/4</code>	$2^{-2} = \frac{1}{4}$
<code>x ∈ ■ et [AB] ⊥ [CD]</code>	x appartient à $\mathbb{R}$ et $[AB]$ perpendiculaire à $[CD]$
<code>A ∪ B et A ∩ B</code>	$A \cup B$ et $A \cap B$

Note : les carrés encore visibles dans la colonne de gauche (l'exposant Unicode « moins » U+207B, le R « double barre » U+211D) sont des symboles que la police de cette colonne ne sait pas rendre — c'est précisément le tofu que le pipeline élimine, comme le montre à chaque fois la colonne de droite.

7. Étape 4 — Police Unicode et génération du PDF

Dernière brique : xhtml2pdf n'embarque que les polices PDF de base (Helvetica...), qui ne couvrent pas $\sqrt{\pi} \leq \geq$. Il faut enregistrer une police TTF du système via ReportLab, avec une liste de candidates ordonnée et un repli, puis la référencer dans le CSS du template :

```
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.pdfmetrics import registerFontFamily
from reportlab.pdfbase.ttf fonts import TTFont
from xhtml2pdf import pisa

# Une police TTF couvrant WGL4, essayée dans l'ordre :
CANDIDATES = [
    r"C:\Windows\Fonts\arialuni.ttf",    # Arial Unicode MS (Office)
    r"C:\Windows\Fonts\seguisym.ttf",    # Segoe UI Symbol (Win10+)
    r"C:\Windows\Fonts\arial.ttf",        # Arial standard
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",    # Linux
]
for path in CANDIDATES:
    if Path(path).exists():
        pdfmetrics.registerFont(TTFont("HakFont", path))
        registerFontFamily("HakFont", normal="HakFont")
        break

html = jinja_env.get_template("rapport.html.j2").render(**contexte)
with open("rapport.pdf", "wb") as f:
    result = pisa.CreatePDF(html, dest=f, encoding="utf-8")
# puis dans le CSS du template : body { font-family: "HakFont"; }
```

Limites CSS de xhtml2pdf à connaître : pas de flexbox ni de grid — mise en page par tableaux ; sous-ensemble CSS 2.1 ; en-têtes/pieds de page via @frame et -pdf-frame-content (utilisés dans ce document même). Pour du contenu scolaire structuré (tableaux de notes, listes, paragraphes), c'est largement suffisant.

8. Checklist d'intégration

1. `pip install "xhtml2pdf>=0.2.14" jinja2 markupsafe`
2. Ajouter la consigne de notation ASCII (section 3) au prompt système de chaque tâche LLM.
3. Créer un module `math_format.py` autonome (regex stdlib uniquement) avec les deux passes de la section 4 — le garder pur le rend partageable entre le PDF et l'UI web, et testable unitairement.
4. Ajouter le filtre WGL4 (section 5) côté rendu PDF.
5. Enregistrer une police TTF Unicode via ReportLab et la déclarer dans le CSS (section 7).
6. Dans le template Jinja2 (autoescape actif), n'injecter les textes maths que via le pipeline complet, marqué Markup : `Markup(math_to_html(_safe_text(texte)))`.
7. Tester avec des cas pièges : dates (06/07/2026), notes (12,5/20), identifiants à underscore, $2^3/2^5$, LaTeX résiduel.

Ordre de grandeur : le module de conversion fait ~140 lignes, le rendu PDF ~400 lignes template compris. Aucun service externe, fonctionne hors-ligne.